

Contents

1	DD08: トークン発行・検証	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	1
1.4.1	3.1 用途別 TTL 一覧	1
1.4.2	3.2 HMAC-SHA256 保存方式	2
1.4.3	3.3 一回限りトークン	2
1.4.4	3.4 再入室トークン	2
1.4.5	3.5 期限切れトークン削除 (cron)	3
1.5	4. 関連設計	3
1.6	5. テスト観点	3
1.7	6. 未確定事項・確認事項	3

1 DD08: トークン発行・検証

1.1 0. 文書情報

項目	内容
文書名	DD08: トークン発行・検証
詳細設計 ID	DD08
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-003 (メール確認) / FR-004 / FR-006 (パスワード再設定) / FR-083 / FR-084 (再入室) / FR-018 (管理者ユーザー有効化)
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-003	メール確認トークン
機能	FR-004 / FR-006	パスワードリセットトークン
機能	FR-018	管理者ユーザー有効化 (activation) トークン
機能	FR-083 / FR-084	再入室トークン (30 日、SCR-027)
API	/me/email-verification/* /auth/password-reset/admin-users/{id}/activation /widget/v1/reentry/*	トークン発行・検証 API
テーブル	access_tokens	トークン基盤

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.5.1	用途別 TTL 一覧	4 用途 × TTL / 一回限り / 内包データ
§ 10.5.2	HMAC-SHA256 保存方式	rawToken 返却 / hash 列保存 / verify / consume
§ 10.5.3	一回限りトークン	consumeToken() 強制
§ 10.5.4	再入室トークン	30 日 / 部屋 reopen 時の新トークン発行

1.4 3. 詳細設計本文

1.4.1 3.1 用途別 TTL 一覧

用途	TTL	一回限り	内包データ
email_verify	24h	◎	accountId
password_reset	1h	◎	accountId
activation	7d	◎	accountId, ownerAccountId
reentry	30d	×	inquiryId

1.4.2 3.2 HMAC-SHA256 保存方式

```
// app/shared/src/lib/token.ts
export async function generateToken(
  purpose: TokenPurpose, payload: Record<string, string>, env: Env, ttlSec: number,
): Promise<string> {
  const tokenId = generateUlid();
  const random = crypto.getRandomValues(new Uint8Array(32));
  const rawToken = `${tokenId}.${base64urlEncode(random)}`;
  const tokenHash = await hmacSha256(env.MASTER_KEY, rawToken);

  await env.DB.prepare(`
    INSERT INTO access_tokens (id, token_hash, purpose, meta, created_at, expires_at, account_id)
    VALUES (?, ?, ?, ?, ?, ?, ?)
  `).bind(tokenId, tokenHash, purpose, JSON.stringify(payload),
    new Date().toISOString(),
    new Date(Date.now() + ttlSec * 1000).toISOString(),
    payload.accountId ?? null).run();

  return rawToken; // ユーザーに返す生トークン (DB には hash のみ)
}

export async function verifyToken(
  rawToken: string, purpose: TokenPurpose, env: Env,
): Promise<{ payload: Record<string, string> }> {
  const tokenHash = await hmacSha256(env.MASTER_KEY, rawToken);
  const record = await env.DB.prepare(
    `SELECT id, purpose, meta, expires_at, used_at FROM access_tokens WHERE token_hash = ?1`
  ).bind(tokenHash).first<TokenRow>();
  if (!record) throw new HTTPException(400, { message: 'TOKEN_INVALID' });
  if (record.purpose !== purpose) throw new HTTPException(400, { message: 'TOKEN_INVALID' });
  if (record.used_at) throw new HTTPException(400, { message: 'TOKEN_REUSED' });
  if (new Date(record.expires_at).getTime() < Date.now()) {
    throw new HTTPException(400, { message: 'TOKEN_EXPIRED' });
  }
  return { payload: JSON.parse(record.meta) };
}

export async function consumeToken(rawToken: string, purpose: TokenPurpose, env: Env) {
  const tokenHash = await hmacSha256(env.MASTER_KEY, rawToken);
  await env.DB.prepare(
    `UPDATE access_tokens SET used_at = ?1 WHERE token_hash = ?2`
  ).bind(new Date().toISOString(), tokenHash).run();
}
```

1.4.3 3.3 一回限りトークン

consumeToken() を検証直後に必ず呼び、used_at を設定。再使用時は TOKEN_REUSED。

1.4.4 3.4 再入室トークン

```
// 再入室トークンは長寿命のため、access_tokens テーブルに `meta.inquiryId` を内包
// 失効・ローテーション運用:
// - チャット部屋作成時に発行
```

// - 30 日経過で `expires_at` に達し失効
 // - 部屋を `closed` → `reopen` した時に新トークン発行 (旧トークンは `expires_at` まで有効)

1.4.5 3.5 期限切れトークン削除 (cron)

`access_tokens` の `expires_at < NOW() - 30` 日を物理削除。詳細は [DD14_ バッチ・非同期処理.md](#) §14.1.9 を参照。

1.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
認証・認可設計	../02_ 基本設計/08_ 認証・認可設計.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD01_ アカウント・ユーザー管理.md / DD06_ 個別チャット.md / DD11_ 暗号化・管理.md

1.6 5. テスト観点

観点	内容
単体	<code>generateToken</code> / <code>verifyToken</code> / <code>consumeToken</code> の正常系 + 異常系全網羅
結合	メール確認 / パスワード再設定 / 管理者ユーザー activation の全 4 用途で発行 → 検証 → consume
異常系	<code>TOKEN_INVALID</code> / <code>TOKEN_REUSED</code> / <code>TOKEN_EXPIRED</code> / <code>TOKEN_PURPOSE_MISMATCH</code>
境界値	TTL ぴったりの境界 (24h / 1h / 7d / 30d) / <code>rawToken</code> の <code>base64url</code> 形式
セキュリティ	DB には <code>hash</code> のみ保存 (生トークン非保存) / <code>HMAC-SHA256</code> + <code>MASTER_KEY</code> 派生
性能	<code>verifyToken</code> は単一 <code>SELECT</code> 、 <code>p95</code> < 50ms

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済